Department of CSE, PES University
COMPUTER NETWORKS-UE24CS252B
CN-MINI-PROJECT

GITHUB_ACCOUNT_LINK:

Secure Online Auction System

What is this project?
This is a **Secure Online Auction System**
Multiple users (clients) connect to a central **server**
Users can **bid on items in real-time**
Communication is **encrypted using SSL**

TEAM MEMBERS DETAILS:
1) PRATHAM KUMAR
SRN: PES2UG24CS368
2) PAVAN E NAIK
SRN: PES2UG24CS343
3) NEEL CHANDRAKAR
SRN: PES2UG24CS310

There are three files in order:

1. **generate_cert.py**
2. **server_ssl.py**
3. **client_ssl.py**

---

1) generate_cert.py

This file creates the SSL certificate and private key for our auction server.

Its job is simple: create a public-private key pair
create a self-signed certificate
save certificate in server_cert.pem
save private key in server_key.pem

The server later uses these two files to run secure SSL communication.

File 1: generate_cert.py — why it exists and how it works
This file is the security setup file. It is not part of the live auction itself. Its only job is to generate the SSL certificate and private key that the server later uses.

Why this file is important

Without this file, the server cannot load certificate credentials for SSL. The server code explicitly loads server_cert.pem and server_key.pem, so these files must already exist before the server is started. That means generate_cert.py is the **foundation of secure communication** in our project.

2) server_ssl.py: This file is the secure auction server. Its job is much bigger:

- run an SSL server
- accept multiple clients
- remember client names
- support two auction types: **Government Auction**
  Private Auction
- send countdowns, rounds, results, and leaderboard
- maintain fairness using thread-safe locking
- keep final auction summary and leaderboard

3) client_ssl.py: This file is the secure auction client. Its logic is:

- ask for server IP and user name
- create SSL connection with the server
- send the client name to server
- listen for messages from server continuously
- based on server messages, update auction state
- allow the user to enter bids
- support both: **Option A: Government Auction**
  Option B: Private Auction
- at the end, display: your auction summary
  final leaderboard
  your rank and wins

Overall purpose of the system: This is a multi-client secure auction platform built using Python sockets, SSL, and threading.
The complete flow is:

- First, the certificate file is generated.
- Then the server starts using that certificate and key.
- Multiple clients connect securely to the server.
- The server announces auction items.
- Clients place bids.
- The server decides winners.
- At the end, both summary and leaderboard are shown.

So the three files are not separate independent programs. They are **three connected parts of one system**.

Why this system is important (Concept-wise)

- Demonstrates **Computer Networks concepts**
- Uses:
  - Socket Programming
  - Client-Server Architecture
  - Concurrency (Threading)
  - Secure Communication (SSL/TLS)
  - Custom Protocol Design

COMPLETE WORKING FLOW (Step-by-Step)

Step 1: Certificate Generation

Run generate_cert.py

Creates secure identity for server

Step 2: Server Starts

Loads certificate and key
Listens on port 5000
Waits for clients

Step 3: Clients Connect

Clients enter IP and name
SSL connection is established
Server stores client info

Step 4: Auction Starts

Server sends: item name
starting price
countdown

Step 5: Bidding Phase

- ✓ **Option A (Government Auction)**
  - Fixed number of rounds
  - One bid per round
  - Highest bid wins
- ✓ **Option B (Private Auction)**
  - Dynamic stages
  - Users choose: continue bidding OR withdraw
  - Last active bidder wins

Step 6: Result Calculation

Server determines: winner
winning bid
profit

Sends result to all clients

Step 7: Final Output

Leaderboard is generated

Each client sees: their rank, number of wins, summary table

HOW SSL WORKS IN THIS PROJECT

SSL (Secure Sockets Layer) ensures:

- **Encryption**
- **Data confidentiality**
- **Secure communication over network**

How SSL is implemented here

Step 1: Certificate creation

generate_cert.py creates: public certificate
private key

These act like server's identity

Step 2: Server setup

Server loads certificate and key

Server becomes secure endpoint

Step 3: Client connection

- Client wraps normal socket with SSL
- Now connection becomes encrypted

What happens internally: When client connects:

Handshake happens : Server sends certificate
Client accepts it (verification is done)

Key exchange: Secure session key is created

Encrypted communication starts: All messages are encrypted. No one can read data in between

---

Why SSL is important here

Prevents: data tampering

Ensures: bids are secure and communication is private

WHAT IS A THREAD?
WHAT IS ITS ROLE HERE?

A thread is a lightweight unit of execution that allows multiple tasks to run concurrently within a program.

Role in your project

On the **server**: Each client is handled in a **separate thread.** So multiple users can bid **at the same time**

On the **client**: One thread → receives messages from server. Main thread → takes user input (bids)

Why needed?

Without threads: One client could block others. Client cannot receive updates while typing. So Threads allow multiple clients to interact with the server simultaneously and ensure real-time communication without blocking.

What is a race condition?

A **race condition** happens when **multiple threads try to access or modify the same shared data at the same time**, and the final result depends on which thread runs first. It is called "race" because the threads are basically racing with each other. In simple words: two or more threads are working together, they touch the same variable or dictionary, their operations overlap, because of that, the data may become wrong or inconsistent.

So the problem is not just "many threads exist." The real problem is: many threads are trying to change the same thing at the same time.

What is a lock? WHY DO WE USE LOCK (THREADING LOCK)?

A **lock** is a synchronization tool that allows **only one thread at a time** to enter a critical section.

Critical section means: the part of code where shared data is being read or modified.

Multiple threads access shared data (bids, leaderboard), Without lock → race condition occurs

Example: Two users bid at same time:

Without lock → data corruption

With lock → safe update

One-line answer: Lock ensures thread-safe access to shared auction data and prevents race conditions.